Bearer token al infinito y más allá

La falsa sensación de seguridad



El conflicto

- **Basic:** resuelve autenticación, no resuelve robo.
- **Cookies:** resuelve estado, no resuelve CSRF.
- **OAuth 2.0:** resuelve delegación, no resuelve posesión.
- **La pregunta:** ¿qué problema resuelve y qué no?

Fuente: RFC 2617; RFC 6265; RFC 6749.

HTTP Basic (1999)

- **Qué resuelve:** autenticación mínima sin estado.
- **Qué no resuelve:** credencial viaja en cada request.
- **Base64 no es cifrado:** está codificado, cualquiera lo lee.
- **Se sigue usando:** sistemas internos, client_secret_basic.

Fuente: RFC 2617 (1999); RFC 7616 (2015).

HTTP Digest (2015)

- **Qué resuelve:** no mandar la contraseña en claro.
- **Qué no resuelve:** el modelo de credencial en cada request.
- **Hash con nonce:** mejora el secreto, no el modelo.
- **Se sigue usando:** prácticamente en desuso.

Fuente: RFC 7616 (2015).

Cookies (1996-2011)

- **Qué resuelve:** recordar quién sos sin repetir la contraseña.
- **Qué no resuelve:** CSRF, robo por XSS, fijación de sesión.
- **Atributos:** HttpOnly, Secure, SameSite.
- **Se sigue usando:** en navegadores web.

Fuente: RFC 6265 (2011).

API keys (2006)

- **Qué resuelve:** identificar aplicaciones sin servidor de identidad.
- **Qué no resuelve:** no expira, no distingue permisos.
- **Si se filtra:** impersonación directa.
- **Se sigue usando:** servicios internos, APIs de terceros.

Fuente: AWS API keys (2006).

SAML 2.0 (2005)

- **Qué resuelve:** SSO, identidad federada, firmas.
- **Qué no resuelve:** XML pesado, difícil en móviles.
- **Se sigue usando:** banca, gobierno, SaaS corporativo.

Fuente: SAML 2.0 (OASIS, 2005).

OAuth 1.0 (2010)

- **Qué resuelve:** request robado no se puede reutilizar.
- **Qué no resuelve:** muy difícil de implementar bien.
- **Firmaba cada request:** HMAC-SHA1 con nonce y timestamp.
- **Se sigue usando:** prácticamente abandonado.

Fuente: RFC 5849 (2010).

Bearer (2012)

- **Qué resuelve:** simplicidad, no firmar cada request.
- **Qué no resuelve:** quien tiene el token, lo usa.
- **La falsa sensación:** bearer no significa seguro, significa portador.

Fuente: RFC 6749; RFC 6750 (2012).

La falsa sensación de seguridad

- **Bearer resuelve:** delegación de autorización.
- **Bearer no resuelve:** autorización, cifrado, anti-replay, MFA.
- **Authorization: Bearer** no significa que la API sea segura.
- **Significa:** quien tiene el token, entra.

Fuente: RFC 6749 §10.3 (2012).

Qué no cubre el bearer

- **No cifra:** JWT es base64, no cifrado.
- **No autoriza:** es otro capítulo.
- **No previene replay:** interceptado = reutilizado.
- **No es MFA:** la autenticación ya pasó.

Fuente: RFC 6749 §10.3 (2012).

JWT (2015)

- **Qué resuelve:** claims autenticables sin estado.
- **Qué no resuelve:** payload no está cifrado.
- **La trampa:** si el validador confía en el alg del token, el atacante elige.

Fuente: RFC 7519 (2015); RFC 8725 (2020).

La línea de tiempo



Fuente: OWASP; IETF; OASIS.

Seguridad dinámica: Step-Up

- **La autenticación no es binaria:** es un flujo continuo.
- **Operaciones sensibles:** exigen re-autenticación.
- **Mecanismo:** RFC 9470, Step-Up Authentication Challenge.

Fuente: RFC 9470 (2023).

Step-Up: flujo 401 y reintento

- **1. La API evalúa:** transacción sensible, acr insuficiente.
- **2. Responde 401:** WWW-Authenticate con acr_values.
- **3. El frontend redirige al IdP** para segundo factor.
- **4. Nuevo token con acr actualizado.** Reintento exitoso.

Fuente: RFC 9470 (2023).

M2M: sin factorizar

- **El invocador es un microservicio** o job de CI.
- **No hay segundo factor:** el equivalente es mTLS.
- **Certificados X.509** o Private Key JWT para firmado de peticiones.

Fuente: RFC 8705 (2020); RFC 7523 (2015).

Apps modificadas

- **Las apps están en manos del atacante:** modificables, descompilables.
- **Riesgo:** cliente Frankenstein que simula ser la app legítima.
- **Desactivan SSL pinning,** extraen secretos, llaman a la API.

Fuente: OWASP Mobile Top 10 (2024).

PKCE: candado del código

- **code_verifier:** string aleatorio generado por el cliente.
- **code_challenge:** hash del verifier, enviado al pedir el código.
- **Al canjear:** el verifier prueba la posesión.
- **Obligatorio hoy** en clientes públicos (RFC 9700).

Fuente: RFC 7636 (2015); RFC 9700 (2025).

App Attestation

- **Android:** Play Integrity API.
- **iOS:** DeviceCheck / App Attest.
- **Verifica el binario no fue modificado** y emite token de attestation.
- **La API lo verifica** antes de operaciones sensibles.

Fuente: Google Play Integrity; Apple DeviceCheck.

Mobile: tres capas

- **App Attestation:** el binario es legítimo.
- **PKCE:** el código no se puede reutilizar.
- **DPoP:** el token no sirve en otro dispositivo.
- **Cliente Frankenstein:** sin binario legítimo, sin PKCE, sin DPoP.

Fuente: Google; Apple; RFC 7636; RFC 9449.

COMPARATIVA

DPoP vs mTLS

Característica	mTLS	DPoP
Capa OSI	Capa 4 (Transporte)	Capa 7 (Aplicación)
Mecanismo	Certificado X.509 en TLS	JWT firmado en header HTTP
Gestión de claves	Compleja (PKI)	Sin PKI, par dinámico
Ideal para	M2M, microservicios	SPAs, móviles

Fuente: RFC 8705 (2020); RFC 9449 (2023).

COMPARATIVA

DPoP: prueba en HTTP

- **Par de claves dinámico:** generado en el momento.
- **Token atado a la clave pública.** La privada permanece en el dispositivo.
- **Header DPoP en cada request:** JWT firmado con método y URL.
- **Funciona a través de proxies, WAFs** y terminaciones TLS.

Fuente: RFC 9449 (2023).

TLS, Gateway, Zero Trust



Fuente: OWASP API Security Top 10 (2023); NIST SP 800-207 (2020).

ID Uruguay

- **OIDC con bearer tokens:** tokens de larga vida sin rotación.
- **SPAs con token en localStorage:** robo por XSS.
- **Mitigaciones:** DPoP, PKCE, Step-Up para operaciones sensibles.

Fuente: AGESIC / ID Uruguay; OWASP.

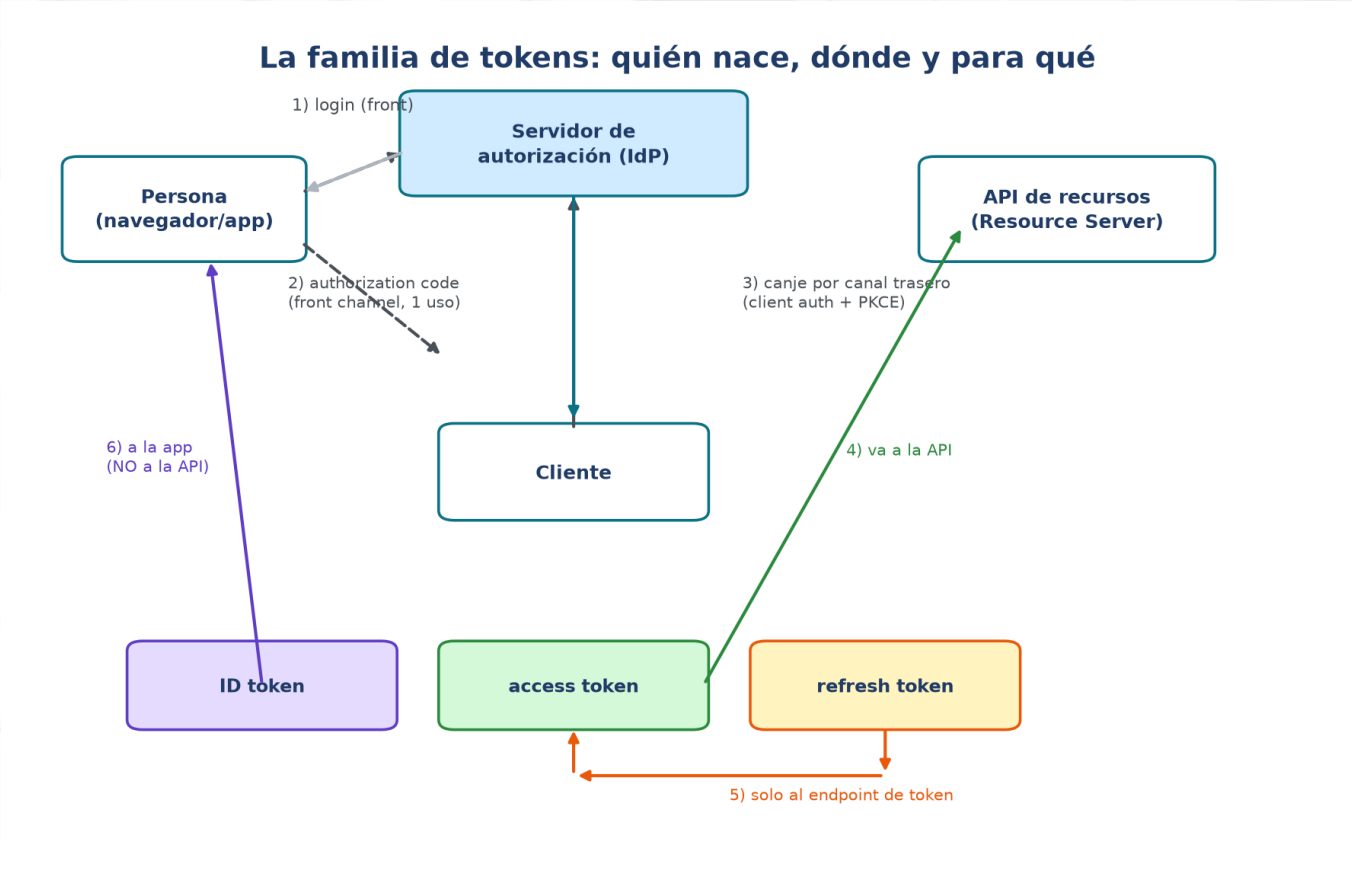
El token se acaba. El riesgo no.

- **La falsa sensación de seguridad:** pensar que bearer significa seguro.
- **Bearer significa:** portador.
- **La seguridad está en el sistema:** autorización, cifrado, anti-replay, MFA.

Fuente: OWASP; RFC 9449; RFC 9700.

DETALLE

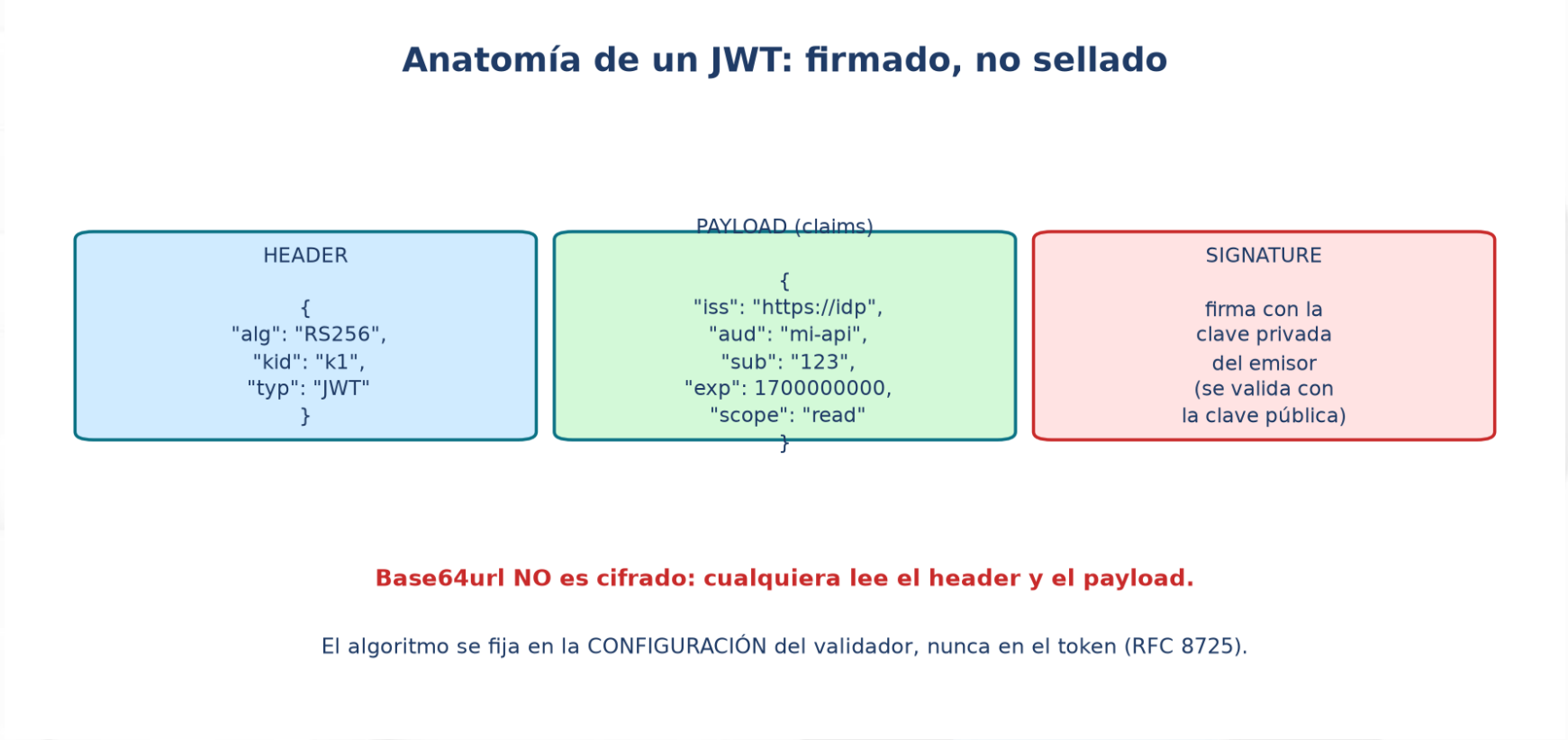
La familia de tokens



Fuente: RFC 6749 §4.1 y §5 (2012); OIDC Core (2014).

DETALLE

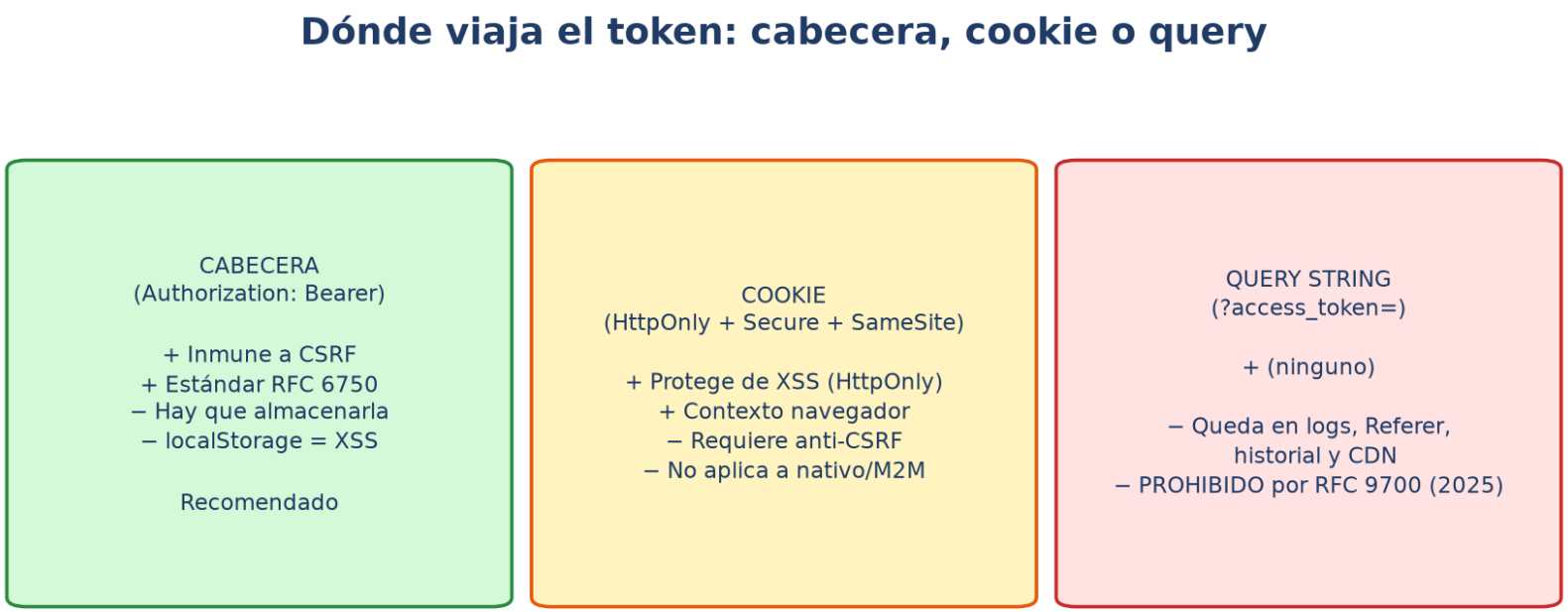
Anatomía de un JWT



Fuente: RFC 7519 (2015); RFC 8725 (2020).

DETALLE

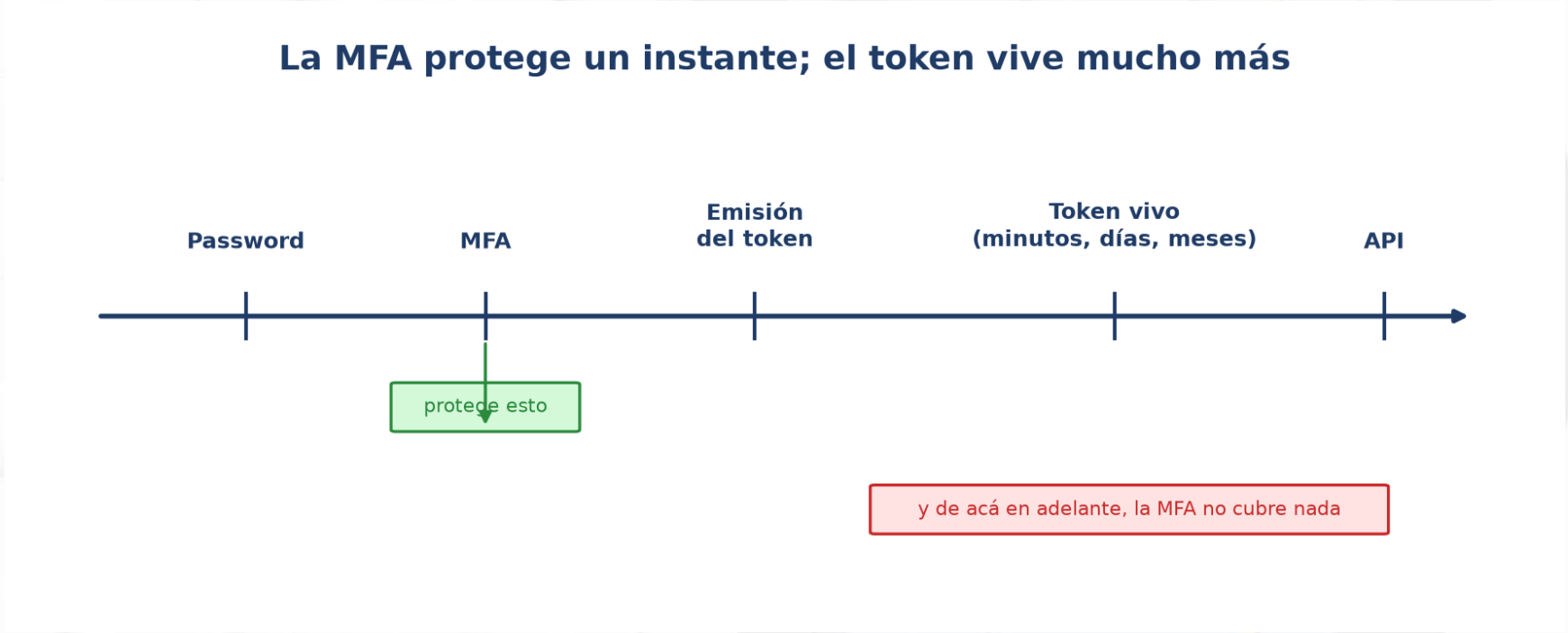
Dónde viaja el token



Fuente: RFC 6750 §2 (2012); RFC 9700 (enero 2025).

DETALLE

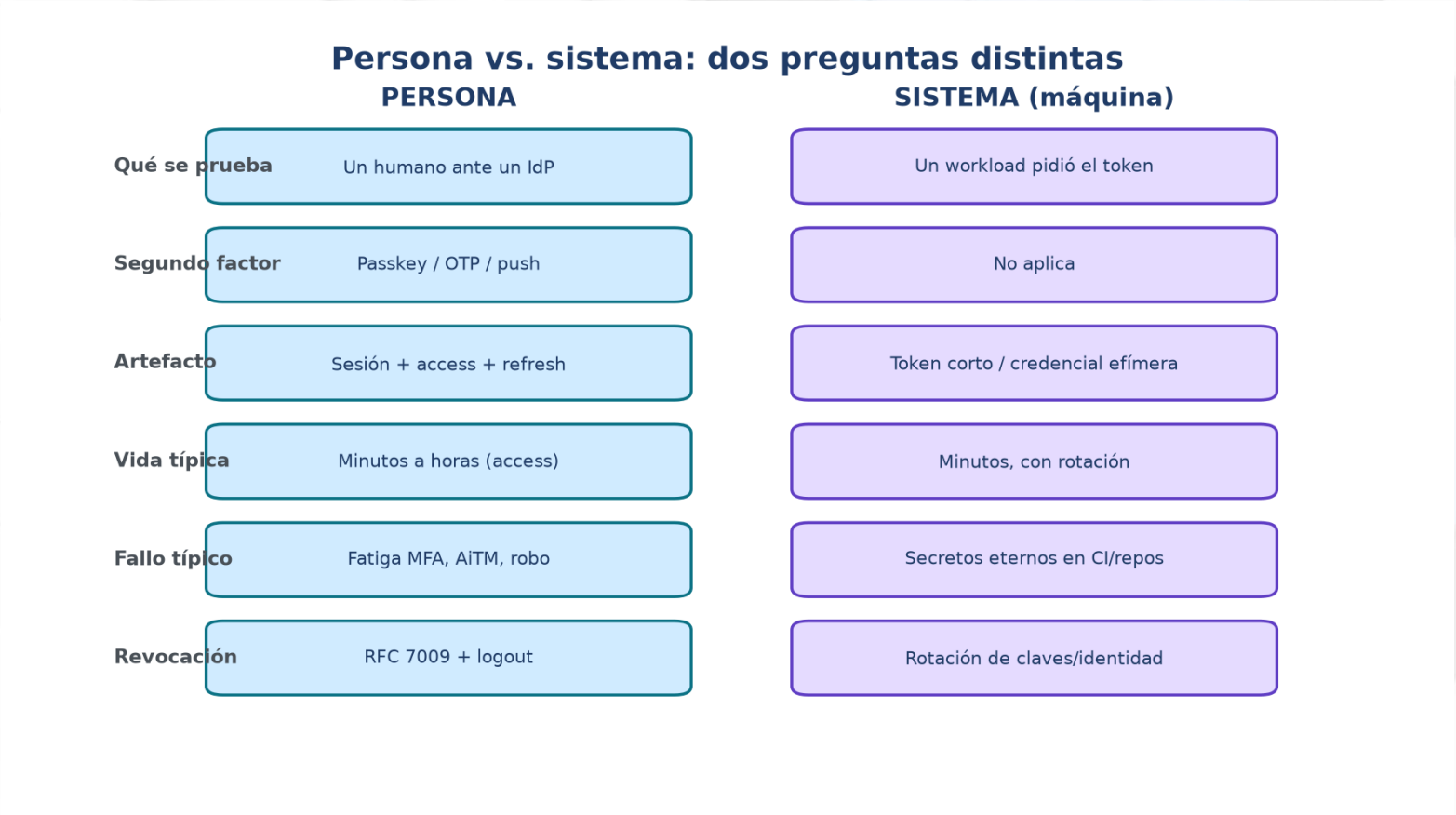
La MFA protege un instante



Fuente: NIST SP 800-63-4 (julio 2025); OIDC Core (2014).

DETALLE

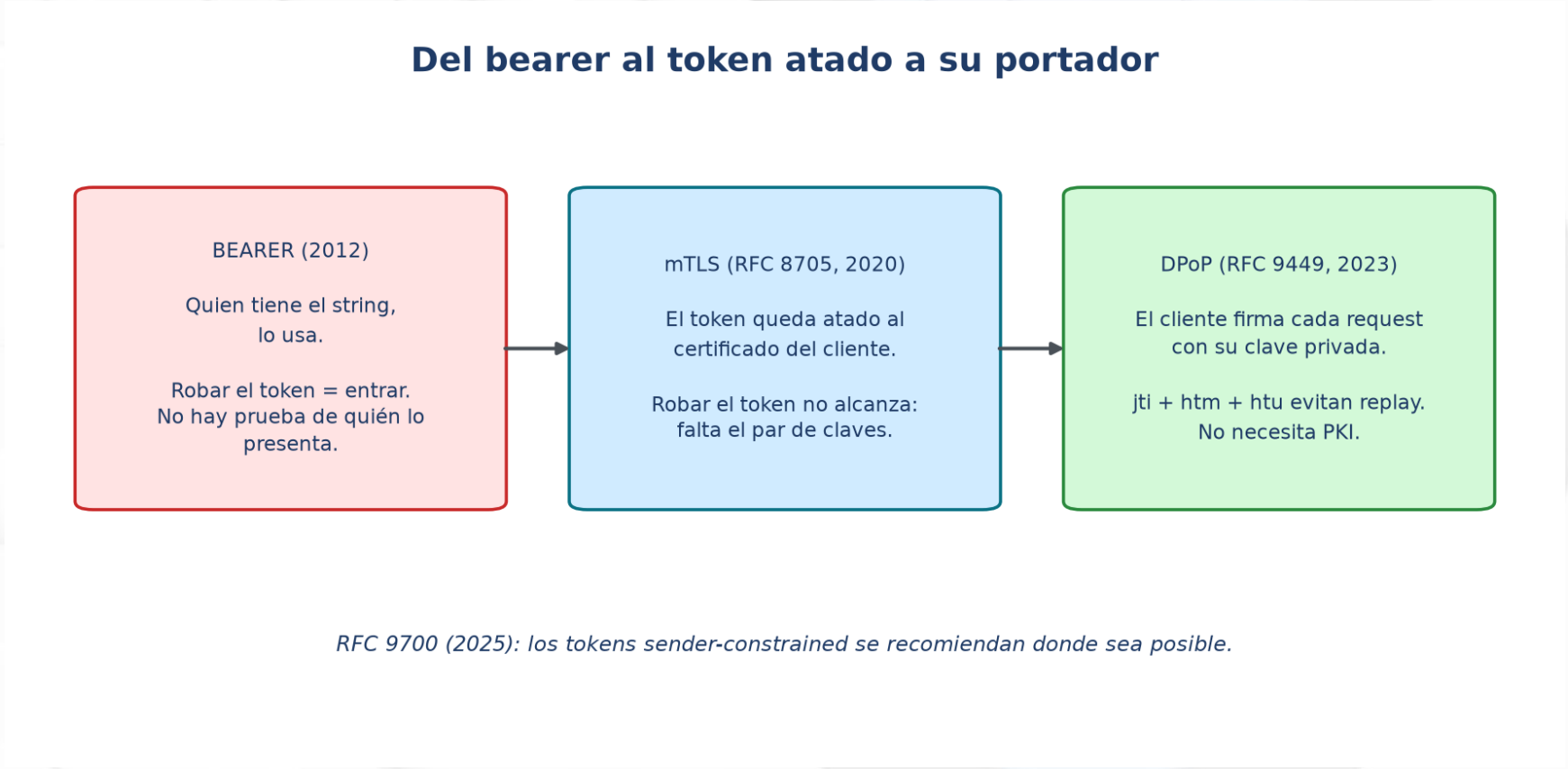
Persona y sistema



Fuente: NIST SP 800-63-4 (2025); GitGuardian (2026).

DETALLE

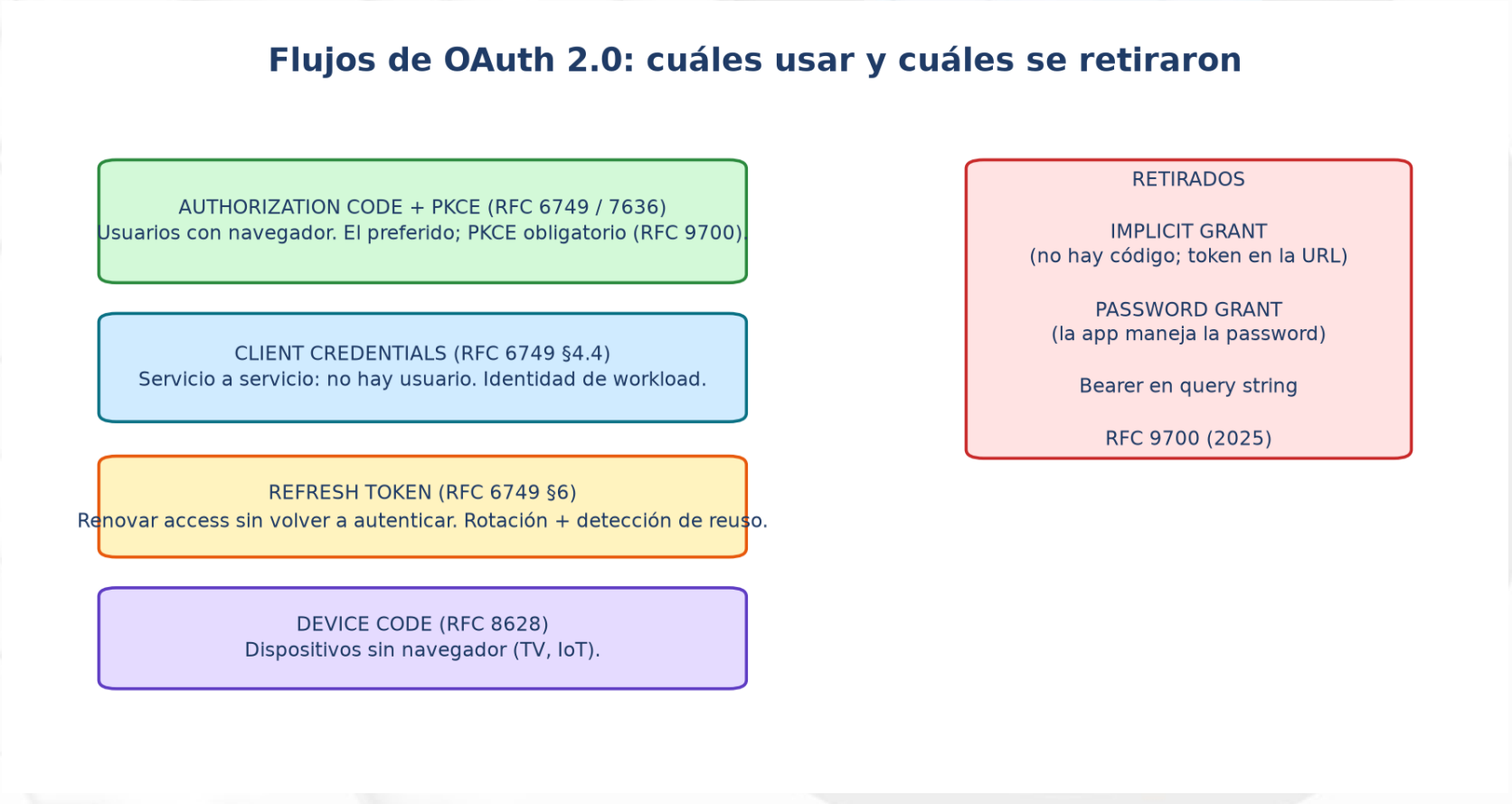
Del bearer al sender-constrained



Fuente: RFC 8471-8473 (2018); RFC 8705 (2020); RFC 9449 (2023).

DETALLE

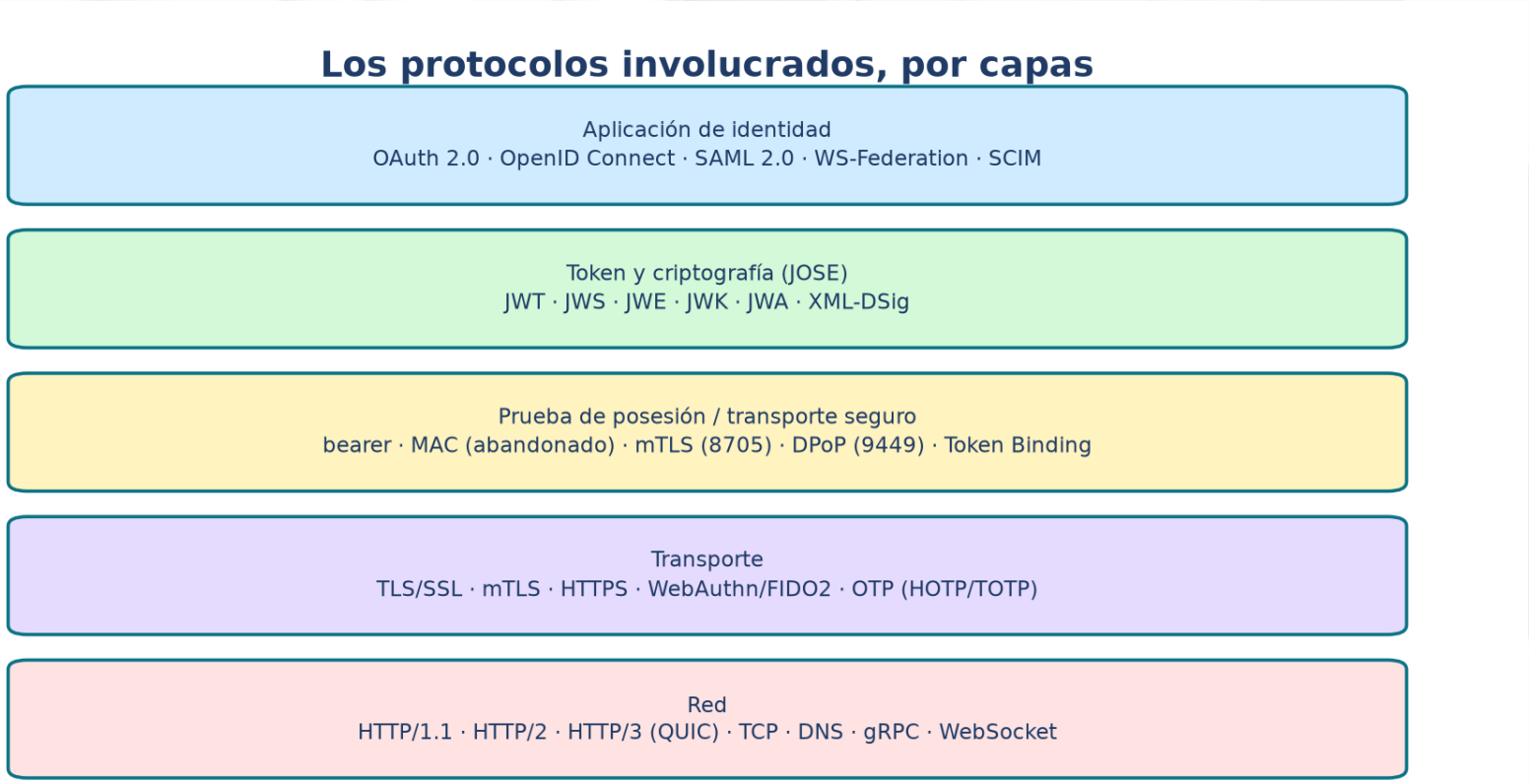
Los flujos de OAuth



Fuente: RFC 6749 (2012); RFC 7636 (2015).

DETALLE

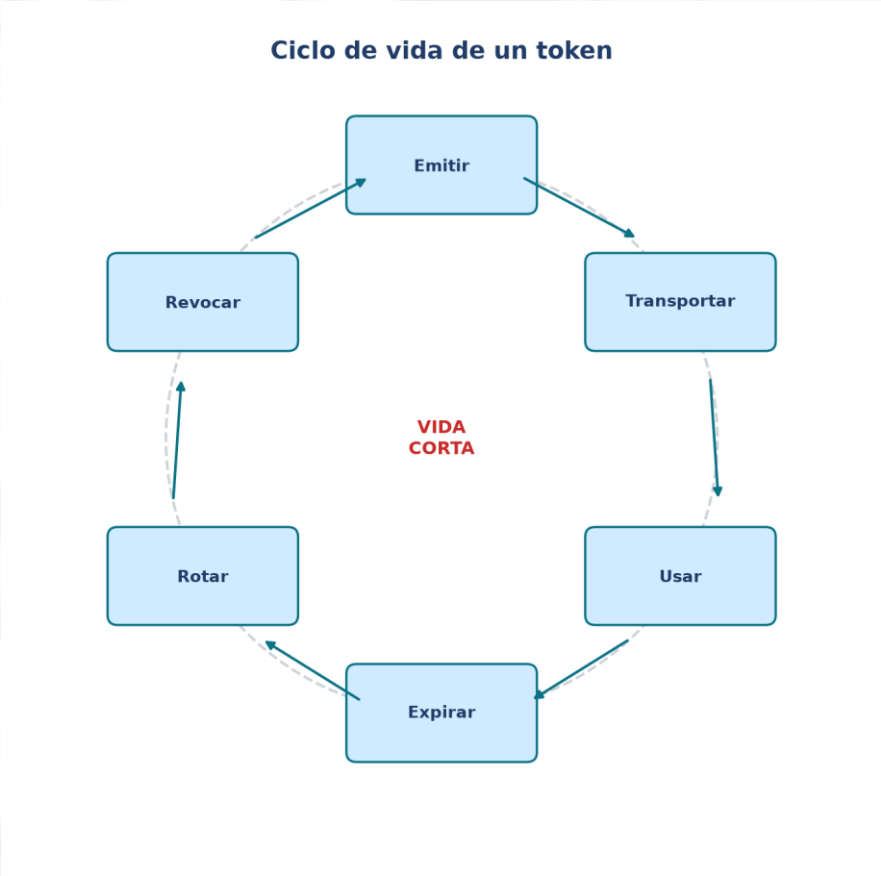
La pila de protocolos



Fuente: OWASP; IETF.

DETALLE

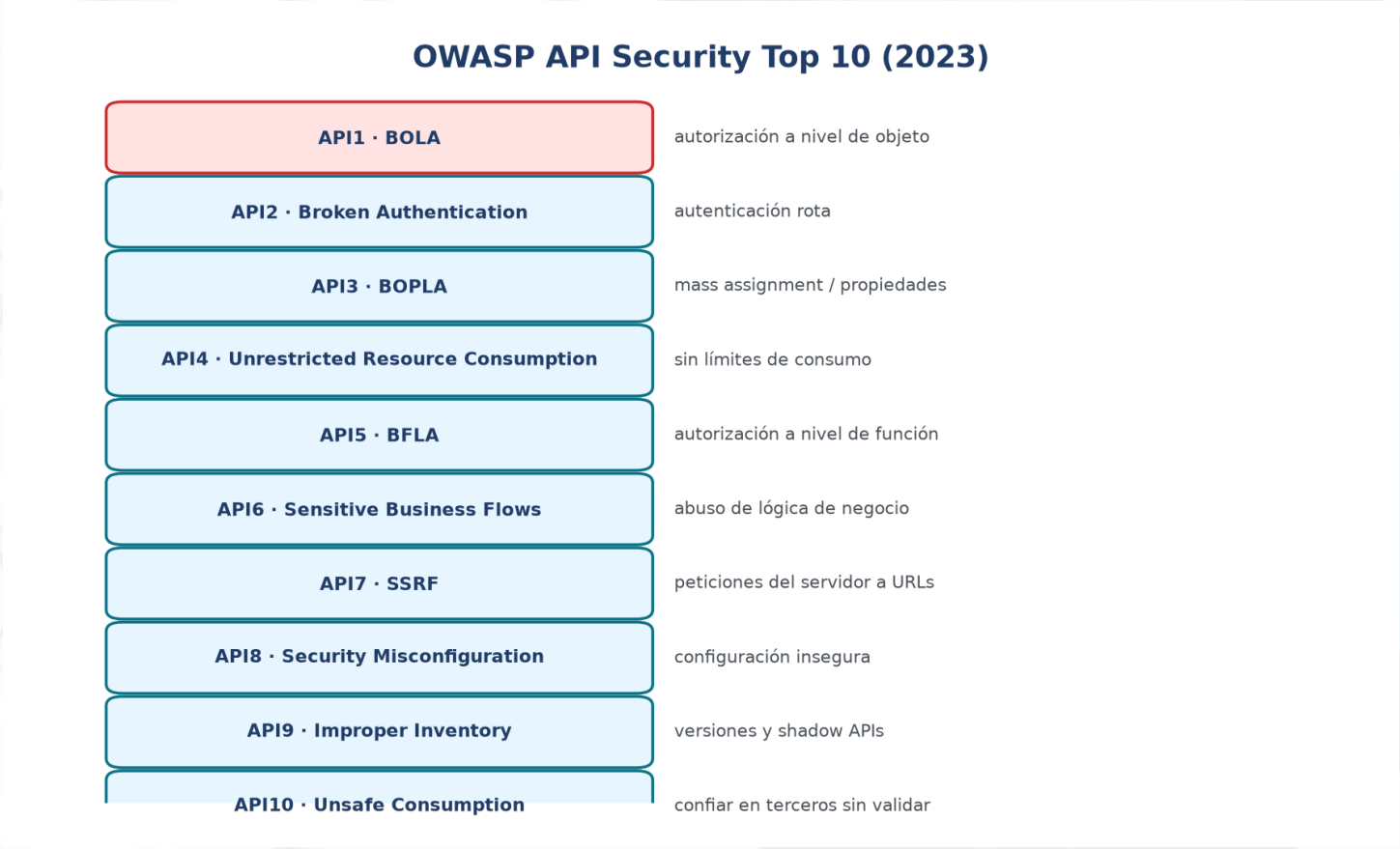
El ciclo de vida del token



Fuente: RFC 6749; RFC 7009; RFC 9700.

DETALLE

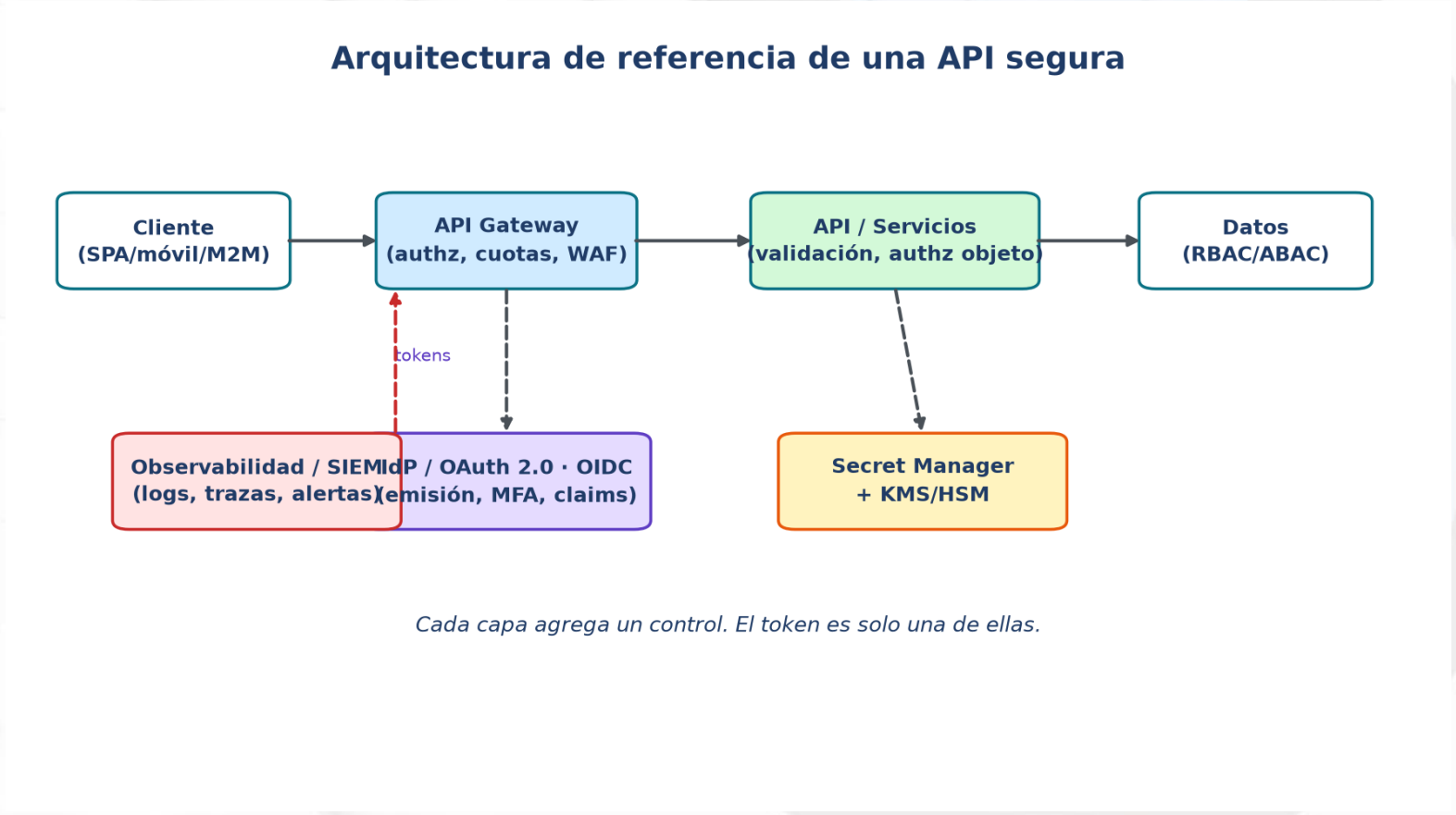
OWASP API Top 10



Fuente: OWASP API Security Top 10, edición 2023.

DETALLE

Arquitectura de referencia



Cada capa agrega un control. El token es solo una de ellas.

Fuente: OWASP; NIST SP 800-207.

DETALLE

El ecosistema completo



Fuente: OWASP API Security Top 10 (2023).

DETALLE

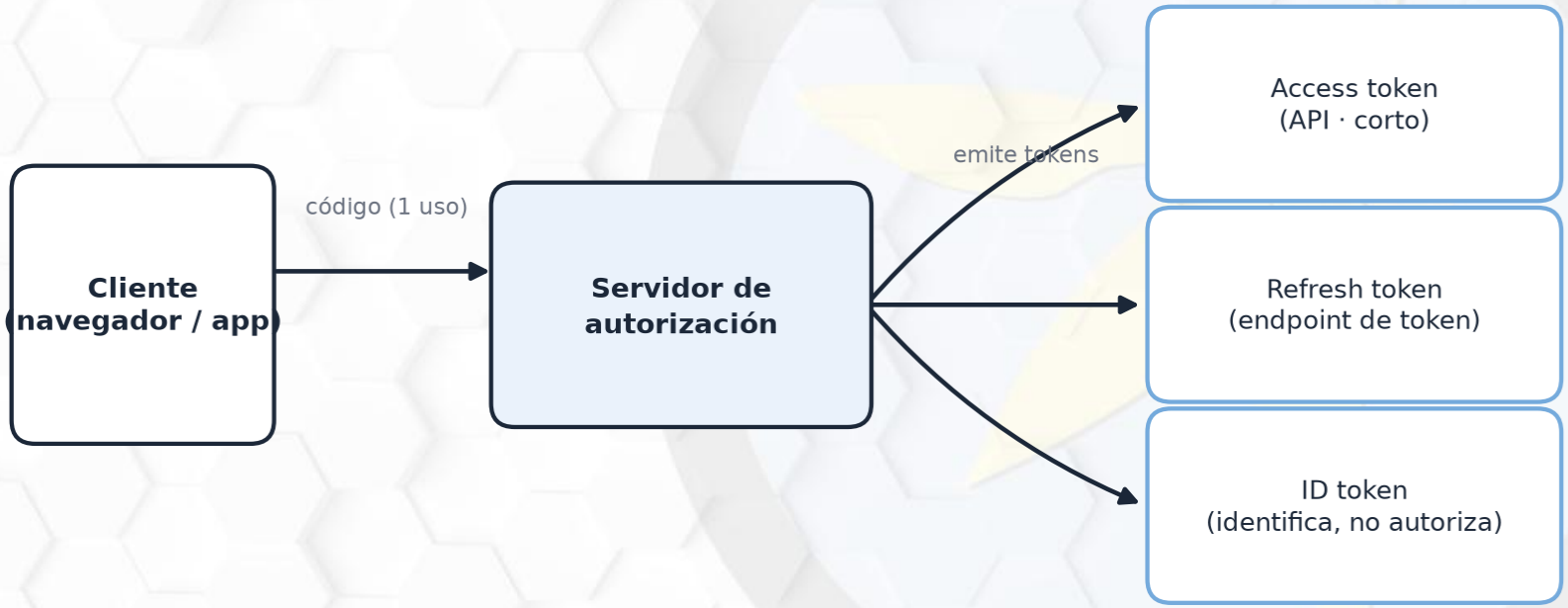
El ecosistema en detalle



Fuente: OWASP API Security Top 10 (2023).

DETALLE

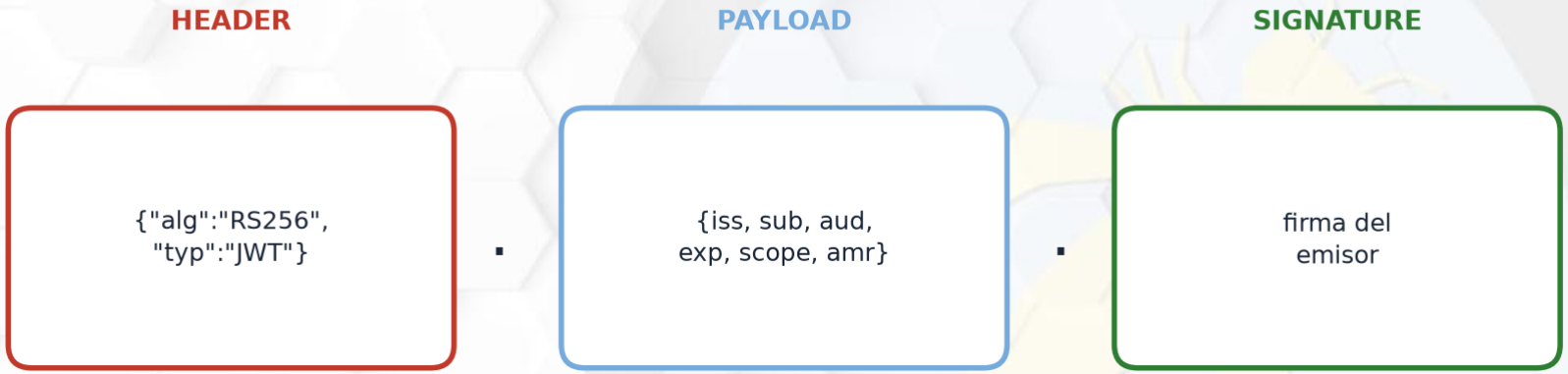
La familia de tokens en detalle



Fuente: RFC 6749 §4.1 y §5 (2012); OIDC Core (2014).

DETALLE

JWT en detalle



cada parte va en base64url · base64 NO es cifrado

firmado ≠ sellado: el payload lo lee cualquiera → JWE para confidencialidad

Fuente: RFC 7519 (2015); RFC 8725 (2020).

DETALLE

MFA en detalle



Fuente: NIST SP 800-63-4 (julio 2025); OIDC Core (2014).

DETALLE

OWASP Top 10 en detalle

Api1 · BOLA — autorización a nivel de objeto

Api2 · Autenticación rota

Api3 · BOPLA — exposición de propiedades

Api4 · Consumo de recursos sin límites

Api5 · BFLA — autorización a nivel de función

Api6 · Acceso a flujos de negocio sensibles

Api7 · SSRF

Api8 · Configuración de seguridad incorrecta

Api9 · Gestión impropia del inventario

Api10 · Consumo inseguro de APIs de terceros

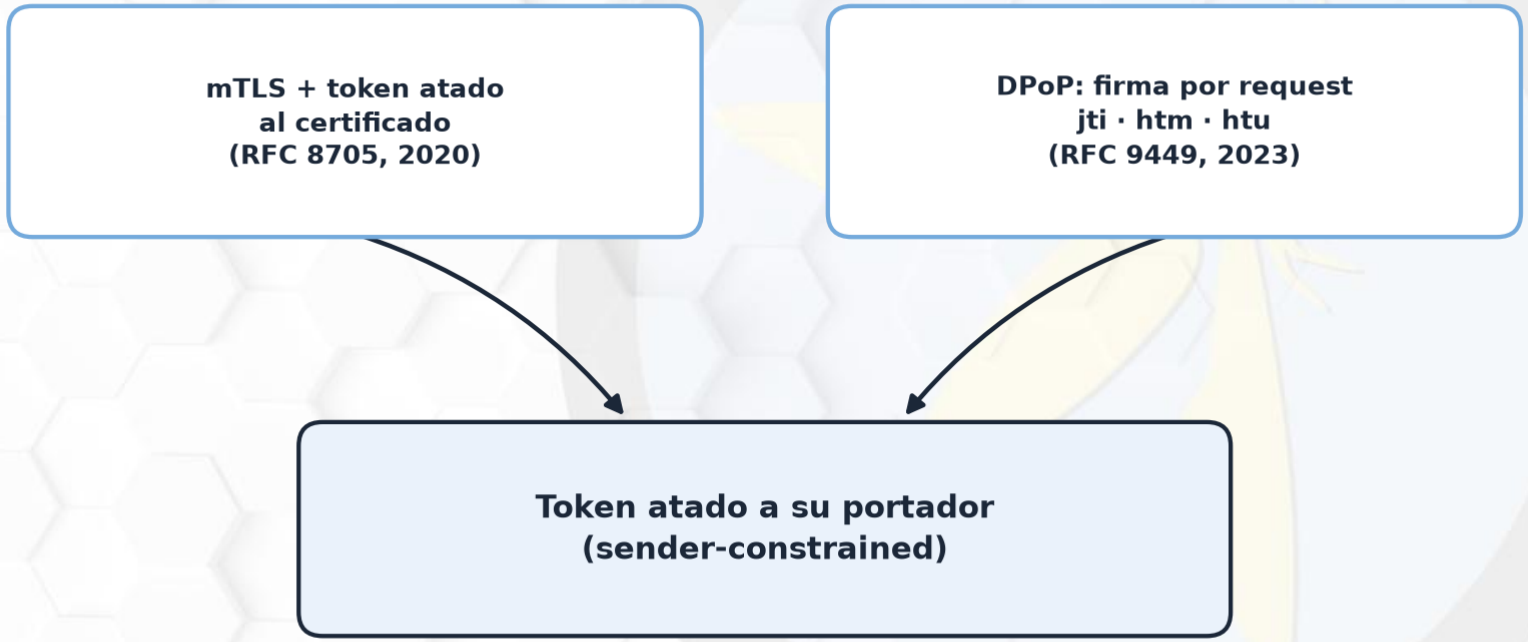
El token es la puerta; estos riesgos viven en todo el edificio.

Fuente: OWASP API Security Top 10, edición 2023.

DETALLE

DPoP en detalle

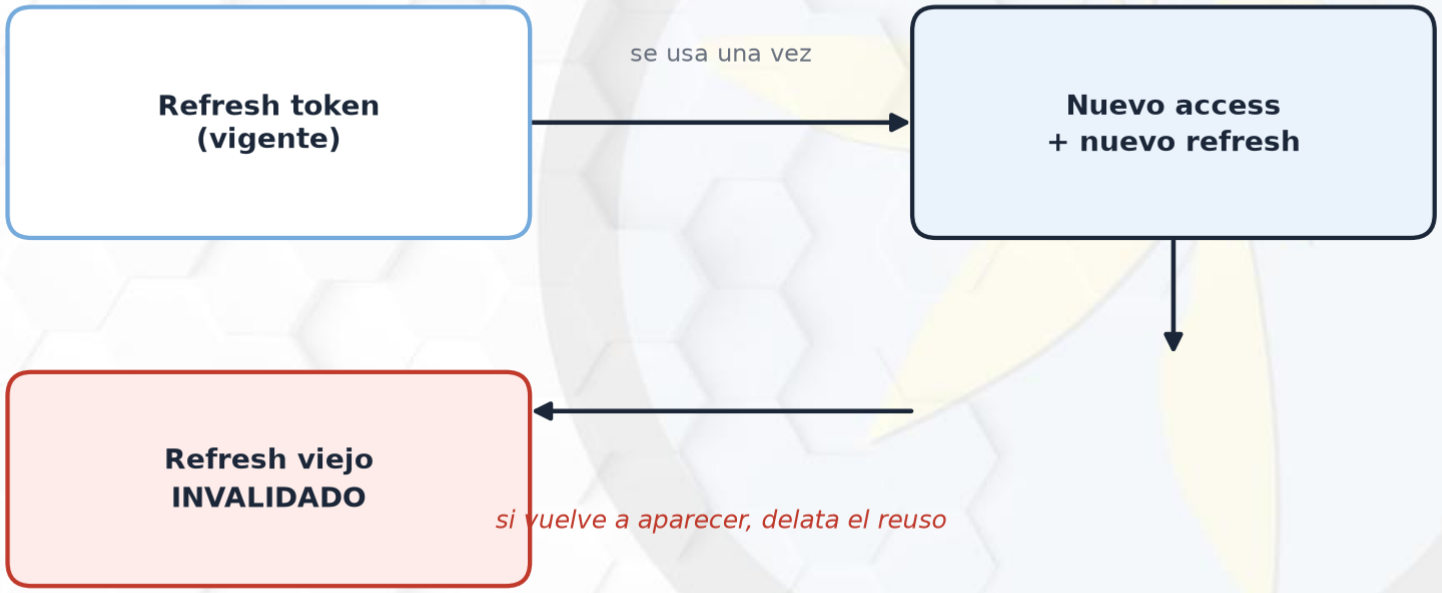
Si la posesión no basta, se exige prueba de posesión.



Fuente: RFC 9449 (2023).

DETALLE

Refresh token en detalle



Fuente: RFC 6749 §10.4 (2012); RFC 7009 (2013); RFC 9700 (2025).

DETALLE

Transporte del token en detalle



inmune a CSRF
(no la envía el navegador sola)

protege de XSS
(exige mitigar CSRF)

prohibido: RFC 9700 (2025)
queda en logs, Referer e historial

Fuente: RFC 6750 §2 (2012); RFC 9700 (enero 2025).